31. Previous Greater Element in Java — Quick Notes

Find the closest greater element on the left side for every element.

Problem Statement

Given an integer array, print the **Previous Greater Element (PGE)** for every element.

Previous Greater Element

For an element at index `i`:
PGE = Closest element on the left that is greater than arr[i]

If no such element exists: PGE = -1

Example

Input: 15 10 18 12 4 6 2 8

Output: -1 15 -1 18 12 12 6 12

Dry Run

Element	Left Side	Previous Greater
15	—	-1
10	15	15
18	15,10	-1
12	15,10,18	18
4	15,10,18,12	12
6	15,10,18,12,4	12
2	...	6
8	...	12

Corner Cases

Increasing Array

Input: 10 20 30 40
Output: -1 -1 -1 -1

No previous greater exists.

Decreasing Array

Input: 40 30 20 10
Output: -1 40 30 20

Immediate left element is always previous greater.

Naive Approach
For each element: Move left until greater element is found.

Complexity

Time	Space
$O(n^2)$	O(1)

Efficient Approach (Monotonic Stack)

Key Observation
We need: Closest Greater Element on the LEFT side.

Which elements become useless?
Suppose stack contains: 30 10 5
Current element: 15
Since:
5 <= 15
10 <= 15

they can never become Previous Greater again.
Remove them.

Stack Property
Maintain a monotonic
decreasing stack.

Stack Stores
ELEMENTS
Unlike Stock Span: Stock Span → store INDEXES
Previous Greater → store VALUES
Because we only need: Previous Greater VALUE

Algorithm
For each element:

Remove smaller/equal elements

```
while(!stack.isEmpty() && stack.peek() <= arr[i]) {
  stack.pop();
}
```

Find Previous Greater

```
if(stack.isEmpty())
  print(-1);
else
  print(stack.peek());
```

Push Current Element
stack.push(arr[i]);

Java Implementation
```
import java.util.ArrayDeque;
import java.util.Deque;

public class PreviousGreater {
  static void printPGE(int arr[]) {

      Deque<Integer> stack = new ArrayDeque<>();
      System.out.print("-1 ");

      stack.push(arr[0]);

      for(int i=1;i<arr.length;i++) {
         while(!stack.isEmpty() && stack.peek() <= arr[i]) {
            stack.pop();
         }
         if(stack.isEmpty())
            System.out.print("-1 ");
         else
            System.out.print(
                  stack.peek() + " ");

         stack.push(arr[i]);
      }
  }

  public static void main(String[] args) {
      int arr[] = {15,10,18,12,4,6,2,8};
      printPGE(arr);
  }
}
```

Dry Run
Input: 15 10 18 12

15
Output: -1
Stack: 15

10
15 > 10
PGE: 15

Push:
10
15

18
Remove:
10
15
Stack becomes empty.

Output: -1
Push: 18

12
18 > 12
PGE: 18

Push:
12
18

Output: -1 15 -1 18

Why O(n)?
Each element:
Pushed once
Popped at most once

Total stack operations:≤ 2n
Therefore: Time Complexity = Θ(n)

Space Complexity
Worst Case
Decreasing array: 40 30 20 10

Stack:
40
30
20
10

Space: O(n)

Best Case
Increasing array: 10 20 30 40
Stack size: 1
Space: O(1)

Relation with Stock Span

Problem	Stack Stores	Formula
Previous Greater	Elements	`stack.peek()`
Stock Span	Indexes	`i - stack.peek()`

Interview Comment (2 lines)

// Maintain a monotonic decreasing stack of previous elements.
// Remove smaller/equal elements; stack top gives the previous greater element.

Quick Revision

Concept	Remember
Goal	Find closest greater element on left
Naive	$O(n^2)$
Efficient	Monotonic Stack
Stack Type	Decreasing Stack
Store	Elements
Remove	Smaller/equal elements
Previous Greater	`stack.peek()`
No Greater Exists	`-1`
Time Complexity	$\Theta(n)$
Auxiliary Space	$O(n)$